1. Git

1. You modified three files in your project, but you want to commit only one of them. Is this possible? How?

Yes. Git allows us to stage and commit only selected files.

git add index.html
git commit -m "Update index page"

Only `index.html` will be included in the commit. The changes in the other files remain uncommitted.

2. What is the difference between `git fetch` and `git pull`? When would you prefer `git fetch`?

- `git fetch` downloads updates from the remote repository but does not merge them into the current branch.
- `git pull` downloads updates and integrates them into the current branch, usually through a merge or rebase depending on the configuration.

Example: I would use `git fetch` when I want to check my teammate's latest changes before integrating them into my own work.

git fetch origin
git log HEAD..origin/main
git merge origin/main

3. You accidentally committed an API key. Is deleting the file and making another commit enough?

No. Deleting the file in a new commit does not remove the API key from earlier commits in Git history.

The appropriate steps are:

1. Revoke or rotate the exposed API key immediately.
2. Remove the secret from the code and store it in environment variables or a secret manager.
3. If necessary, rewrite Git history to remove the secret from previous commits.
4. Coordinate with collaborators and clean up affected copies of the repository.

Important: Even if the secret is removed from Git history, it should be treated as compromised.

4. Two developers modify the same line and push their changes to the same branch. What problem might occur?

A merge conflict may occur when Git cannot automatically combine the changes.

To resolve it:

1. Pull or fetch the latest changes.
2. Open the conflicting file.
3. Decide which changes to keep or combine.
4. Remove Git conflict markers.
5. Test the code and commit the resolution.

Git conflict markers look like this:

<<<<<<< HEAD
Your changes
=======
Teammate's changes
>>>>>>> feature-branch

5. You committed changes locally, but your teammate cannot see them on GitHub. Why?

The changes may have been committed locally but not pushed to the remote repository.

git push origin main

A commit saves changes in the local repository. A push uploads those commits to the remote repository, such as GitHub.

Other possible reasons include pushing to the wrong branch or repository.

2. HTML

1. What is HTML, and what is its role in web development?

HTML stands for HyperText Markup Language. It defines the structure and content of a web page using elements such as headings, paragraphs, images, links, and forms.

HTML provides structure, CSS controls presentation, and JavaScript adds interactivity.

2. What are semantic HTML elements? Give examples.

Semantic elements describe the meaning or purpose of their content.

Examples include:

- `<header>` – introductory content or a header.
- `<nav>` – navigation links.
- `<main>` – the main content of the page.
- `<article>` – self-contained content.
- `<section>` – a thematic section.
- `<footer>` – footer information.

They improve accessibility, code readability, and the ability of search engines to understand page structure.

3. What is the difference between `<div>` and `<span>`?

- `<div>` is a generic block-level container by default. It typically starts on a new line.
- `<span>` is a generic inline container by default. It typically stays within the current line.

Both can be used to group content for styling or JavaScript manipulation.

4. What is the purpose of the `alt` attribute in images?

The `alt` attribute provides alternative text describing an image.

<img src="student.jpg" alt="Student working on a laptop">

It helps screen-reader users understand meaningful images and provides fallback text when an image cannot load. Decorative images can generally use an empty `alt=""`.

5. What is the difference between `id` and `class`?

- `id` identifies a particular element and should be unique within the HTML document.
- `class` can be shared by multiple elements.

<h1 id="main-title">Welcome</h1>
First paragraph
Second paragraph

In CSS, IDs use `#`, while classes use `.`.

6. What are block-level and inline elements?

- Block-level elements generally begin on a new line and occupy the available width. Examples: `<div>`, `<p>`, and `<section>`.
- Inline elements generally remain within the same line and occupy the space their content requires. Examples: `<span>`, `<a>`, and `<strong>`.

CSS can change an element's display behavior.

7. What is the use of the `<form>` tag?

The `<form>` element groups controls used to collect and submit user input.

```
<form>
  <label for="email">Email:</label>
  <input type="email" id="email" name="email" required>
  <button type="submit">Submit</button>
</form>
```

Forms are commonly used for login, registration, search, and feedback.

8. What is the difference between `<strong>` and `<b>`?

- `<strong>` indicates that its content has strong importance.
- `<b>` draws attention to text without indicating the same level of importance.

Both are typically displayed in bold by default, but their semantic meanings differ.

9. What are attributes in HTML?

Attributes provide additional information or configuration for HTML elements. They are generally written in the opening tag.

```
<a href="https://example.com" target="_blank">Visit website</a>
```

Here, `href` specifies the destination and `target` specifies where the link should open.

10. What is the purpose of `<meta>` tags?

`<meta>` elements provide metadata about an HTML document, usually inside `<head>`.

Examples:

```
<meta charset="UTF-8">
```

<meta name="viewport" content="width=device-width, initial-scale=1.0">
<meta name="description" content="Learn web development">

They can specify character encoding, viewport settings for responsive layouts, and a page description for search engines.

3. CSS

1. What is CSS, and why do we use it?

CSS stands for Cascading Style Sheets. It controls the appearance and layout of web pages.

It is used to define colors, fonts, spacing, alignment, animations, and responsive layouts.

2. What is the difference between inline, internal, and external CSS?

- Inline CSS: Written directly in an HTML element using the `style` attribute.
- Internal CSS: Written inside a `<style>` element in the HTML document.
- External CSS: Written in a separate `.css` file and linked to the HTML document.

Example:

```
<!-- Inline -->
<p style="color: red;">Hello</p>

<!-- Internal -->
<style>
  p { color: blue; }
</style>

<!-- External -->
<link rel="stylesheet" href="style.css">
```

External CSS is generally preferred for maintainability and reuse across multiple pages.

3. What is the CSS box model?

The CSS box model describes how the browser calculates the space occupied by an element.

It consists of four layers:

1. Content: The text, image, or other content.
2. Padding: Space between the content and border.

3. Border: The boundary around the padding and content.
4. Margin: Space outside the border.

By default, the declared `width` and `height` apply to the content box. Using `box-sizing: border-box` includes padding and border within the specified dimensions.

4. What is the difference between margin and padding?

- `margin` creates space outside an element's border.
- `padding` creates space between the content and the element's border.

For example, use padding to create space inside a button and margin to create space between two buttons.

5. What is the difference between `display: none` and `visibility: hidden`?

- `display: none` hides the element and removes its layout space.
- `visibility: hidden` hides the element but preserves its layout space.

Both hide the element visually, but their effects on layout differ.

6. What is Flexbox, and why is it used?

Flexbox is a CSS layout system designed to arrange elements along a row or column.

It helps align, distribute, and space elements efficiently, especially when the available space changes.

```
.container {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 16px;
}
```

Here, `justify-content` distributes items along the main axis, while `align-items` aligns them along the cross axis.

7. What is the `position` property in CSS? Name its types.

The `position` property controls how an element is positioned in a document.

The main values are:

- `static` – normal document flow; the default.
- `relative` – remains in normal flow and can be offset from its original position.
- `absolute` – removed from normal flow and positioned relative to its containing block.
- `fixed` – positioned relative to the viewport in typical cases.
- `sticky` – behaves like a relatively positioned element until a scrolling threshold is reached, then sticks within its scrolling context.

Properties such as `top`, `right`, `bottom`, and `left` can control offsets where applicable.

8. What is `z-index`?

`z-index` controls the stacking order of overlapping elements in applicable stacking contexts.

An element with a higher `z-index` is generally painted above one with a lower value within the same stacking context.

```
.modal {
  position: fixed;
  z-index: 1000;
}
```

A high `z-index` does not guarantee that an element appears above every other element because stacking contexts also matter.

9. What is responsive design?

Responsive design allows a website to adapt to different screen sizes and devices.

It uses techniques such as:

- Flexible layouts with Flexbox or Grid.
- Relative units such as `%` and `rem`.
- Responsive images.
- Media queries.

```
@media (max-width: 600px) {
  .container {
    flex-direction: column;
  }
}
```

10. What is the difference between a class selector and an ID selector?

A class selector targets elements that share a class, while an ID selector targets an element with a particular ID.

```
.card {
  padding: 16px;
}

#header {
  background-color: black;
}
```

Class selectors use `.`, and ID selectors use `#`. ID selectors generally have higher specificity than class selectors.

4. Tailwind CSS

1. What is Tailwind CSS?

Tailwind CSS is a utility-first CSS framework that provides small, reusable utility classes for styling HTML elements.

For example:

```
<button class="bg-blue-600 text-white px-4 py-2 rounded">
  Submit
</button>
```

These classes apply a background color, text color, horizontal and vertical padding, and rounded corners.

2. How is Tailwind different from traditional CSS?

Traditional CSS usually involves writing custom styles and applying them through selectors. Tailwind lets developers apply predefined utility classes directly in markup.

Traditional CSS:

```
<button class="submit-btn">Submit</button>
.submit-btn {
  background-color: blue;
  color: white;
  padding: 8px 16px;
}
```

Tailwind CSS:

```
<button class="bg-blue-600 text-white px-4 py-2">
  Submit
</button>
```

Tailwind can speed up styling and maintain consistency, although long class lists can make markup harder to read.

3. What does utility-first CSS mean?

Utility-first CSS means building a design by combining small classes, each responsible for a specific style.

Examples:

- `p-4` – padding.
- `text-center` – centered text.
- `font-bold` – bold text.
- `flex` – flex layout.
- `rounded-lg` – rounded corners.

4. How do you add margin and padding in Tailwind?

Tailwind uses `m` for margin and `p` for padding.

Class	Meaning
`m-4`	Margin on all sides
`mt-4`	Margin on top
`mx-4`	Horizontal margin

`p-4`	Padding on all sides
`px-4`	Horizontal padding
`py-2`	Vertical padding

The numeric values refer to Tailwind's spacing scale, not necessarily the same number of pixels.

5. What is `flex` in Tailwind?

The `flex` utility applies `display: flex` to an element.

```
<div class="flex items-center justify-between gap-4">
  <span>Logo</span>
  <button>Login</button>
</div>
```

- `items-center` aligns items along the cross axis.
- `justify-between` distributes items along the main axis.
- `gap-4` adds spacing between items.

6. How do you make layouts responsive in Tailwind?

Tailwind provides breakpoint prefixes such as `sm:`, `md:`, `lg:`, and `xl:`.

```
<div class="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3">
  ...
</div>
```

This produces one column by default, two columns at the medium breakpoint, and three at the large breakpoint.

By default, these responsive prefixes apply at their respective minimum-width breakpoints.

7. What does `hover:` do?

The `hover:` prefix applies a utility when the user hovers over an element.

```
<button class="bg-blue-600 hover:bg-blue-800 text-white">
  Submit
</button>
```

The button's background becomes darker when hovered.

8. What is the use of the `container` class?

The `container` utility sets a responsive maximum width at configured breakpoints.

It is useful for keeping content within a readable width on larger screens.

```
<div class="container mx-auto px-4">
  Page content
</div>
```

`mx-auto` centers the container horizontally when there is available space. The `container` utility does not automatically add all desired padding or centering in every configuration.

9. What are `bg-`, `text-`, and `border-` classes?

These prefixes apply different styling utilities.

- `bg-blue-500` – background color.
- `text-white` – text color.
- `text-xl` – text size.
- `border-2` – border width.
- `border-gray-300` – border color.

The exact available utilities and values depend on the Tailwind version and configuration.

10. What are the advantages of Tailwind CSS?

- Faster UI development.
- Consistent spacing, colors, and typography.
- Responsive styling through breakpoint utilities.
- Less need to write custom CSS for common layouts.
- Easy customization through configuration and theme variables.

A limitation is that HTML can become difficult to read when many utility classes are combined in one element.

5. JavaScript

1. What are the different data types in JavaScript? How do primitive types differ from reference types?

JavaScript has seven primitive types:

- `string`
- `number`
- `bigint`
- `boolean`
- `undefined`
- `symbol`
- `null`

Objects are non-primitive values. Arrays and functions are also objects or object-like callable values in JavaScript.

Primitive values are copied as values. Objects are accessed through references, so assigning an object to another variable copies the reference rather than creating an independent copy of the object.

```
let a = 10;
let b = a;
b = 20;

console.log(a); // 10

const user1 = { name: "Alex" };
const user2 = user1;

user2.name = "Sam";

console.log(user1.name); // Sam
```

In the second example, both variables refer to the same object.

2. What is a closure? Give an example of using a closure to create a private variable.

A closure occurs when a function retains access to variables from its surrounding lexical scope, even after the outer function has finished executing.

```
function createCounter() {
  let count = 0;
```

```
  return function () {
    count++;
    return count;
  };
}

const counter = createCounter();

console.log(counter()); // 1
console.log(counter()); // 2
```

The `count` variable cannot be accessed directly from outside `createCounter()`. The returned function can still access and update it.

This pattern is useful for maintaining private state.

3. What is the Temporal Dead Zone (TDZ) in relation to `let` and `const`?

The Temporal Dead Zone is the period between entering a scope and initializing a `let` or `const` variable.

Accessing the variable during this period causes a `ReferenceError`.

```
console.log(age); // ReferenceError
let age = 25;
```

Although the declaration is known to JavaScript, the variable cannot be accessed before initialization. `const` has the same TDZ behavior and must be initialized when declared.

4. How does a `switch` statement differ from a series of `if-else` blocks in readability and performance?

A `switch` statement is useful when comparing one expression against several possible values.

```
switch (role) {
  case "admin":
    console.log("Full access");
    break;
  case "student":
    console.log("Student access");
    break;
  default:
    console.log("Unknown role");
}
```

An `if-else` chain is more flexible when conditions involve ranges, multiple variables, or complex Boolean expressions.

For performance, neither is universally faster. The difference depends on the code and JavaScript engine; readability and suitability should generally guide the choice.

5. What are truthy and falsy values in JavaScript? List at least five falsy values.

Truthy values behave like `true` in Boolean contexts. Falsy values behave like `false`.

The falsy values are:

- `false`
- `0`
- `-0`
- `0n`
- `""` (empty string)
- `null`
- `undefined`
- `NaN`

Additionally, `document.all` is a legacy browser-specific exception.

Example:

```
if ("hello") {
  console.log("This executes");
}

if (0) {
  console.log("This does not execute");
}
```

Empty arrays `[]` and empty objects `{}` are truthy in JavaScript.

6. What is the difference between a `for...in` loop and a `for...of` loop?

- `for...in` iterates over enumerable property keys, usually object property names.
- `for...of` iterates over values provided by an iterable, such as an array or string.

```
const fruits = ["apple", "mango"];
```

```
for (const index in fruits) {
  console.log(index); // "0", "1"
}

for (const fruit of fruits) {
  console.log(fruit); // "apple", "mango"
}
```

For ordinary array iteration, `for...of` is generally the clearer choice when the values are needed.

7. How do `break` and `continue` affect a loop?

- `break` immediately terminates the nearest enclosing loop or switch.
- `continue` skips the rest of the current loop iteration and moves to the next iteration.

```
for (let i = 1; i <= 5; i++) {
  if (i === 2) continue;
  if (i === 4) break;

  console.log(i);
}
```

Output:

```
1
3
```

8. What is hoisting? Does it apply to functions and variables in the same way?

Hoisting describes how JavaScript processes declarations before executing code within a scope.

Different declarations behave differently:

- Function declarations: Can generally be called before their declaration.
- `var`: The declaration is hoisted and initialized to `undefined`.
- `let` and `const`: They are hoisted but cannot be accessed before initialization because of the TDZ.
- Function expressions: Their behavior depends on the variable used to hold the function.

```
console.log(value); // undefined
var value = 10;
```

```
sayHello(); // Hello

function sayHello() {
  console.log("Hello");
}
```

9. What is callback hell, and how do Promises and async/await solve this problem?

Callback hell occurs when multiple asynchronous operations are nested inside callbacks, making code difficult to read, maintain, and handle errors in.

A nested callback example:

```
getUser(function (user) {
  getOrders(user.id, function (orders) {
    getPayment(orders[0].id, function (payment) {
      console.log(payment);
    });
  });
});
```

Promises let asynchronous operations be chained using `.then()` and errors handled using `.catch()`.

Async/await provides a more sequential-looking way to write Promise-based code.

```
async function loadData() {
  try {
    const user = await getUser();
    const orders = await getOrders(user.id);
    const payment = await getPayment(orders[0].id);

    console.log(payment);
  } catch (error) {
    console.error(error);
  }
}
```

Both approaches improve readability and error handling. `async/await` does not automatically make operations parallel; `Promise.all()` can be used when independent operations should run concurrently.

10. What is the use of the nullish coalescing operator (??)?

The `??` operator returns the right-hand value only when the left-hand value is `null` or `undefined`.

```
const username = null ?? "Guest";
console.log(username); // Guest

const count = 0 ?? 10;
console.log(count); // 0
```

Unlike `||`, it preserves valid falsy values such as `0`, `false`, and `""`.

6. React

1. What is the Virtual DOM? Explain reconciliation.

The Virtual DOM is a commonly used term for React's in-memory representation of the UI.

When state or props change, React renders the component tree again and reconciles the new result with the previous one. It uses element types, keys, and tree position to determine which parts of the UI can be reused and which need updating.

React then commits the necessary changes to the real DOM.

The process avoids requiring developers to manually update every affected DOM element, although it does not mean every React update is automatically faster than direct DOM manipulation.

2. When would you choose a Class Component over a Functional Component today?

For new React development, Functional Components with Hooks are generally recommended.

A Class Component may still be appropriate when:

- Maintaining an existing application that already uses classes.
- Working within a codebase whose conventions favor class components.
- Maintaining a class-based error boundary, since React's traditional error-boundary API uses class lifecycle methods.

Class components are still supported, but they are not generally the first choice for new components.

3. Explain one-way data flow in React. Can a child modify a prop?

React follows a one-way data flow: data typically moves from parent components to child components through props.

A child should not directly modify a prop it receives. Instead, the parent can pass a callback that the child invokes to request a change.

```
function Parent() {
  const [name, setName] = useState("Alex");

  return <Child name={name} onChangeName={setName} />;
}

function Child({ name, onChangeName }) {
  return (
    <button onClick={() => onChangeName("Sam")}>
      {name}
    </button>
  );
}
```

The parent owns the state, and the child requests an update through the callback.

4. What is the significance of the dependency array in `useEffect`? What happens if you return a function?

The dependency array determines when an effect should run again.

```
useEffect(() => {
  console.log("Runs after each committed render");
});
useEffect(() => {
  console.log("Runs when dependencies change");
}, [userId]);
useEffect(() => {
  console.log("Runs after the initial mount");
}, []);
```

The empty-array example describes the usual behavior in production, with development Strict Mode potentially performing an additional setup-and-cleanup cycle.

If the effect returns a function, React treats it as a cleanup function. Cleanup runs before the effect is set up again when dependencies change and when the component unmounts.

```
useEffect(() => {
  const timer = setInterval(() => {
    console.log("Running");
  }, 1000);

  return () => clearInterval(timer);
}, []);
```

This prevents the interval from continuing after cleanup.

5. What is a Single Page Application (SPA), and how does React enable it?

A Single Page Application is a web application that updates its displayed content without requiring a full-page reload for every navigation.

React helps build SPAs by rendering and updating UI components in the browser. A routing library such as React Router can map URLs to different page components while preserving the SPA experience.

A React application can also be configured for server-side rendering or multi-page behavior, so React itself does not require every application to be an SPA.

6. What are lifecycle methods? Name their functional equivalents using `useEffect`.

Lifecycle methods are methods associated with different stages of a Class Component's life.

Important examples:

- `componentDidMount()` – called after the component mounts.
- `componentDidUpdate()` – called after an update.
- `componentWillUnmount()` – called before unmounting.

In Functional Components, `useEffect` can perform similar side effects:

- `useEffect(() => { ... }, [])` – setup after mounting, with development Strict Mode caveats.
- `useEffect(() => { ... }, [value])` – setup when the component mounts and when `value` changes, with cleanup before re-running.
- A cleanup function returned from an effect – cleanup before re-running or unmounting.

These are conceptual equivalents, not exact one-to-one replacements for every lifecycle method.

7. What is the difference between Controlled and Uncontrolled Components? Which is recommended?

A Controlled Component has its form value managed by React state.

```
const [email, setEmail] = useState("");

<input
  value={email}
  onChange={(e) => setEmail(e.target.value)}
/>
```

An Uncontrolled Component keeps its current value in the DOM, often accessed through a ref.

```
const inputRef = useRef(null);

<input ref={inputRef} defaultValue="" />
```

Controlled inputs are useful when the UI must respond to input changes, validate as the user types, or keep form state synchronized with other components.

Uncontrolled inputs can be simpler for basic forms. Neither approach is universally best; choose according to the requirements.

8. What is prop drilling, and how does the Context API solve it?

Prop drilling occurs when data is passed through several intermediate components even though only a deeply nested component needs it.

React Context lets a component provide a value to descendants without explicitly passing that value through every intermediate component.

```
const ThemeContext = createContext("light");

function App() {
  return (
    <ThemeContext.Provider value="dark">
      <Page />
    </ThemeContext.Provider>
  );
}

function Page() {
  const theme = useContext(ThemeContext);
  return <p>Current theme: {theme}</p>;
```

}

Context is useful for shared values such as themes, authenticated-user information, or locale settings. It should not automatically replace props for all data.

9. Explain the concept of a Slice.

In React application development, particularly with Redux Toolkit, a slice represents a portion of the application state and the logic used to update it.

A slice usually contains:

- A name.
- An initial state.
- Reducers describing state updates.
- Automatically generated action creators.

```
import { createSlice } from "@reduxjs/toolkit";

const counterSlice = createSlice({
  name: "counter",
  initialState: { value: 0 },
  reducers: {
    increment(state) {
      state.value += 1;
    },
    decrement(state) {
      state.value -= 1;
    }
  }
});

export const { increment, decrement } = counterSlice.actions;
export default counterSlice.reducer;
```

Redux Toolkit uses Immer so reducer code can express updates using this convenient syntax while producing immutable state updates.

10. How do you handle dynamic URL parameters, such as an ID, using React Router?

React Router supports dynamic route segments using a colon prefix.

```
<Route path="/students/:id" element={<StudentDetails />} />
```

Inside the component, `useParams()` retrieves the value:

```
import { useParams } from "react-router-dom";

function StudentDetails() {
  const { id } = useParams();

  return <h1>Student ID: {id}</h1>;
}
```

For the URL `/students/123`, the `id` parameter will be `"123"`.

11. What are Pure Components?

Pure Components are components that avoid unnecessary re-rendering when their props and state have not changed according to a shallow comparison.

In React:

- `React.PureComponent` provides this behavior for class components.
- `React.memo()` can provide similar prop-based memoization for functional components.

```
const Student = React.memo(function Student({ name }) {
  return <p>{name}</p>;
});
```

`React.memo()` may skip a re-render when the props are shallowly equal. It does not guarantee that a component never re-renders, and it does not prevent re-renders caused by the component's own state or consumed context changes.

7. Python Viva Questions

Variables, Data Types and Operators

1. What is a variable in Python? How is variable declaration different from languages such as C or Java?

A variable is a name that refers to a value or object in memory.

In Python, we do not need to explicitly declare a variable's data type. The type is determined by the value assigned to it.

age = 25name = "Alex"price = 99.5

In C or Java, variables generally require explicit type declarations, such as `int age = 25;` in Java.

Python is dynamically typed, meaning a variable can later refer to a value of a different type.

value = 10value = "Hello"

2. What are the main built-in data types in Python?

Common built-in data types include:

Data type	Example
`int`	`10`
`float`	`10.5`
`complex`	`2 + 3j`
`str`	`"Hello"`
`list`	`[1, 2, 3]`
`tuple`	`(1, 2, 3)`
`set`	`{1, 2, 3}`

`dict`	`{"name": "Alex"}`
`bool`	`True`, `False`
`NoneType`	`None`

Lists, dictionaries, and sets are mutable. Tuples, strings, integers, and floats are examples of immutable objects.

3. What is the difference between mutable and immutable data types?

Mutable objects can be modified after they are created, while immutable objects cannot be modified after creation.

Example of a mutable object:

numbers = [10, 20, 30]numbers[0] = 100print(numbers)# [100, 20, 30]

Example of an immutable object:

name = "Python"name = name + " Programming"print(name)# Python Programming

In the string example, the original string is not modified. A new string is created and assigned to `name`.

Common mutable types include `list`, `dict`, and `set`. Common immutable types include `int`, `float`, `str`, and `tuple` (although a tuple can contain references to mutable objects).

4. What is the difference between `/`, `//`, `%`, and `**` operators?

These are arithmetic operators in Python.

Operator	Purpose	Example	Result

`/`	Division	`10 / 3`	`3.333...`
`//`	Floor division	`10 // 3`	`3`
`%`	Remainder	`10 % 3`	`1`
`**`	Exponentiation	`2 ** 3`	`8`

Floor division rounds the quotient down toward negative infinity, rather than simply removing the decimal part.

For example:

print(10 // 3) # 3print(-10 // 3) # -4

Control Statements

The following questions are marked for Python 11:30 students in the supplied document.

5. What is the difference between `if`, `if-else`, and `if-elif-else` statements?

- `if` executes a block when a condition is true.
- `if-else` selects between two alternative blocks.
- `if-elif-else` checks multiple conditions in sequence and executes the first matching branch.

Example:

marks = 75if marks >= 90:   print("Grade A")elif marks >= 70:   print("Grade B")else: print("Grade C")

Output:

Grade B

Python uses indentation to define the blocks belonging to each condition.

6. What is the difference between a `for` loop and a `while` loop?

A `for` loop is generally used to iterate over a sequence or iterable, such as a list, string, or range.

A `while` loop repeats as long as its condition remains true.

Example of a `for` loop:

for number in range(1, 4): print(number)

Example of a `while` loop:

number = 1while number <= 3: print(number) number += 1

Both examples produce:

1
2
3

A `while` loop requires careful condition management to avoid an unintended infinite loop.

7. What is the difference between `break`, `continue`, and `pass`?

- `break` immediately terminates the nearest enclosing loop.
- `continue` skips the remaining statements in the current iteration and proceeds to the next iteration.
- `pass` performs no operation and serves as a placeholder when a statement is syntactically required.

Example:

for number in range(1, 6): if number == 2: continue if number == 4: break
print(number)

Output:

1
3

Example of `pass`:

```
def future_feature():
    pass
```